

TP - Session 16 : Manipulation des Selectors

Organiser l'accès au state Redux

Module M203 - Développement Front-End

Objectif : Créer une application e-commerce avec des selectors organisés.

Contexte du Projet

Vous allez créer une application e-commerce avec :

- Une liste de **produits** (id, name, price, category, stock)
- Un **panier** (productId, quantity)

State prévu :

```
{
  products: {
    items: [
      { id: 1, name: 'Laptop', price: 999, category: 'tech', stock: 5 },
      { id: 2, name: 'T-Shirt', price: 29, category: 'clothing', stock: 50 },
      { id: 3, name: 'Casque Audio', price: 150, category: 'tech', stock: 0 },
      { id: 4, name: 'Chaussures', price: 89, category: 'clothing', stock: 20 },
    ]
  },
  panier: {
    items: [
      { productId: 1, quantity: 2 },
      { productId: 4, quantity: 1 }
    ]
  }
}
```

1 Exercice 1 : Selectors Simples (4 pts)

Fichier : src/features/products/productsSelectors.js

Créer les selectors simples suivants :

<code>selectProducts</code> : retourne <code>state.products.items</code>	(1 pt)
<code>selectProductsCount</code> : retourne le nombre de produits	(1 pt)
<code>selectPanierItems</code> : retourne <code>state.panier.items</code>	(1 pt)
<code>selectPanierIsEmpty</code> : retourne <code>true</code> si le panier est vide	(1 pt)

2 Exercice 2 : Selectors avec Filtrage (4 pts)

Fichier : src/features/products/productsSelectors.js

Créer les selectors suivants :

<code>selectAvailableProducts</code> : retourne les produits avec <code>stock > 0</code>	(2 pts)
<code>selectTechProducts</code> : retourne les produits de categorie <code>'tech'</code>	(2 pts)

3 Exercice 3 : Selector avec Tri (3 pts)

Fichier : `src/features/products/productsSelectors.js`

Créer le selector `selectProductsSortedByPrice` qui retourne les produits triés par prix croissant.

Attention

N'oubliez pas de copier le tableau avec `[...array]` avant de trier pour ne pas modifier le state original !

4 Exercice 4 : Selector avec Calcul (4 pts)

Fichier : `src/features/panier/panierSelectors.js`

Créer le selector `selectPanierTotal` qui calcule le prix total du panier.

- Pour chaque item du panier, trouver le produit correspondant
- Multiplier le prix par la quantité
- Retourner la somme totale

5 Exercice 5 : Selector avec Parametre (3 pts)

Fichier : `src/features/products/productsSelectors.js`

Créer le selector `selectProductById` qui accepte un id en parametre et retourne le produit correspondant.

6 Exercice 6 : Utilisation dans un Composant (2 pts)

Fichier : `src/components/ProductList.jsx`

Créer un composant qui :

- Importe et utilise `selectAvailableProducts`
- Affiche la liste des produits disponibles (nom et prix)

Bonus (+3 pts)

- Créer `selectProductsByCategory(state, category)` avec parametre
- Créer `selectPanierDetails` qui retourne les items avec infos produit (name, price, sub-total)
- Ajouter un composant de recherche utilisant `selectProductsBySearch`

Exercice 2

Contexte : Application de gestion de produits avec Redux.

1. Qu'est-ce qu'un **selector** dans Redux ? (2 pts)
 2. Donner **2 avantages** d'utiliser des selectors plutot que d'ecrire la logique directement dans le composant. (2 pts)
 3. Donner le code du selector simple **selectProducts** qui retourne **state.products.items**. (2 pts)
 4. Donner le code du selector **selectAvailableProducts** qui retourne les produits dont le **stock > 0**. (3 pts)
 5. Donner le code du selector **selectProductsSortedByPrice** qui retourne les produits tries par prix croissant. Expliquer pourquoi on utilise **[...array]**. (3 pts)
 6. Donner le code du selector **selectTotalStock** qui retourne la somme des stocks de tous les produits (utiliser **reduce**). (3 pts)
 7. Donner le code du selector **selectProductById** qui accepte un **id** en parametre. (2 pts)
 8. Donner le code d'un composant **ProductCount** qui utilise le selector **selectProductsCount** pour afficher le nombre de produits. (3 pts)
-

Note : Les 3 types de Selectors

- **Simple** : `(state) => state.products.items`
- **Avec logique** : `(state) => items.filter(...)` ou `sort()` ou `reduce()`
- **Avec parametre** : `(state, id) => items.find(p => p.id === id)`